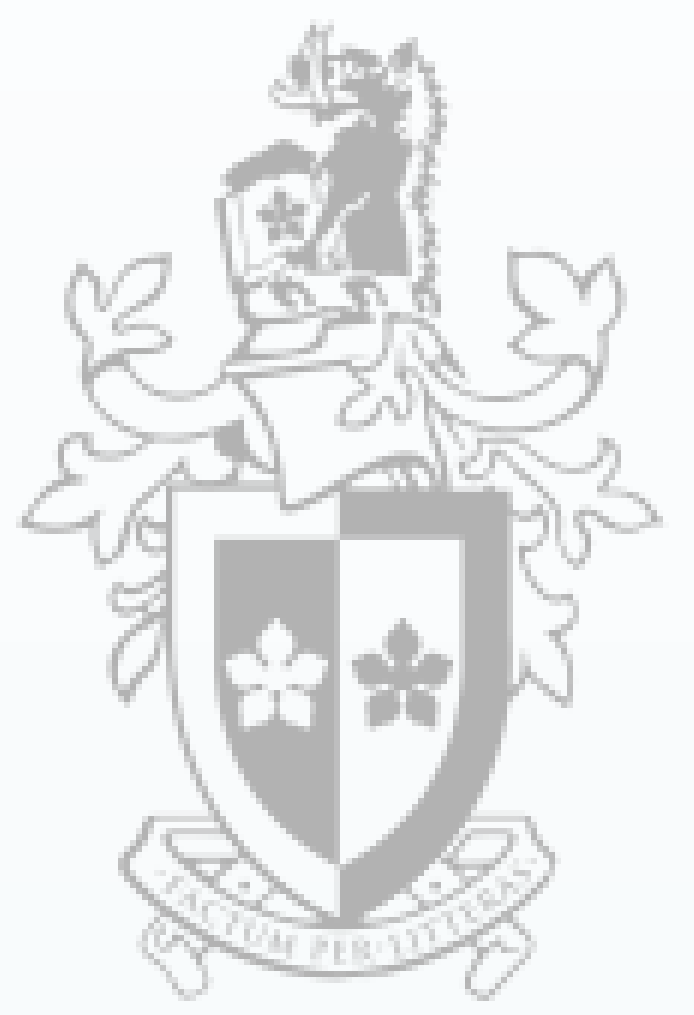


Database Schema Comparison and Version Control System

Umer Naseer, Frandya Meidy F., Cindy Wong IY | Supervised by: Jason Thomas Chew

Faculty of Engineering, Computing and Science
Swinburne University of Technology Sarawak Campus

Email: JTChew@swinburne.edu.my



Introduction

Database schema drift occurs when environments silently diverge, leaving undocumented changes to reach production as missing columns, wrong types, or absent constraints that crash applications.

This tool automates PostgreSQL schema drift detection, generates SQL migration scripts, and commits every versioned change to GitHub, replacing a fragile manual process with a structured, auditable workflow.

Research Objectives

- Compare two live PostgreSQL schemas and surface all structural differences
- Match renamed entities via weighted similarity scoring
- Auto-generate SQL migration scripts from detected differences
- Assign SemVer versions (Major.Minor.Patch), auto-bumped by change severity
- Commit scripts to GitHub with a pull-and-apply deployment pipeline

Problem Statement

Schema changes made in development often go undocumented and reach production as missing columns, wrong types, or absent constraints. The client experienced this firsthand: an untracked change caused a live production outage.

Core Challenges

- No automated visibility into structural differences between environments
- Changes tracked only in developer memory, no formal record
- Scripts written manually, prone to omission and inconsistency
- Renames appear as drop-and-create without similarity analysis
- No version history tied to the database or environment

Scope

Compares tables, columns, types, order, nullability, PKs, and unique/FK/check/exclude constraints.

Literature Review

Schema Matching

Rahm & Bernstein (2001) show that effective schema matching must combine name, structural, and semantic signals, informing the weighted scoring algorithm in this tool.

Melnik et al. (2002) show that entity similarity propagates to structurally related neighbours, motivating the column-context weighting in the matching phase.

Schema Evolution

Brahmia et al. (2024) find that automated migration tooling remains fragmented and largely requires manual script authoring: the gap this project addresses.

Hu et al. (2022) show that properly sequenced schema changes can be applied with near-zero downtime, motivating the version-gapped execution model in the apply pipeline.

Existing Tools: Gap Analysis

Flyway and Liquibase manage script execution and ordering, but neither detects drift nor generates scripts automatically. This project fills that upstream gap.

System Architecture

The tool introspects two PostgreSQL instances via information_schema and pg_catalog, runs weighted similarity matching, computes a colour-coded diff, generates a migration script, pushes it to GitHub, and provides a pull-and-apply pipeline for the target database.

References

- [1] Rahm, E. & Bernstein, P. A. (2001). A survey of approaches to automatic schema matching. VLDB Journal, 10(4), 334–350.
- [2] Melnik, S., Garcia-Molina, H. & Rahm, E. (2002). Similarity Flooding. Proc. 18th ICDE. IEEE.
- [3] Brahmia, Z., Grandi, F. & Oliboni, B. (2024). A Literature Review on Schema Evolution in Databases. Computing Open, 2(1).
- [4] Hu, T. et al. (2022). Online Schema Evolution is (Almost) Free for Snapshot Databases. PVLDB, 16(2), 140–153.

Methodology

A three-phase pipeline: compare two live schemas, generate a versioned migration script, apply it to the target database.

01

Schema Comparison

- Introspect two live PostgreSQL schemas via information_schema + pg_catalog
- Score every table pair and column pair using weighted similarity
- Handles renames: structural match beats exact-name miss
- Output: colour-coded diff (added / removed / modified)

02

SQL Generation & Versioning

- Translate diff into a valid SQL migration script
- Auto-assign SemVer: Major (DROP), Minor (ADD), Patch (modify)
- Script is editable before committing
- Push to GitHub via Contents API under /scripts/vX.Y.Z.sql

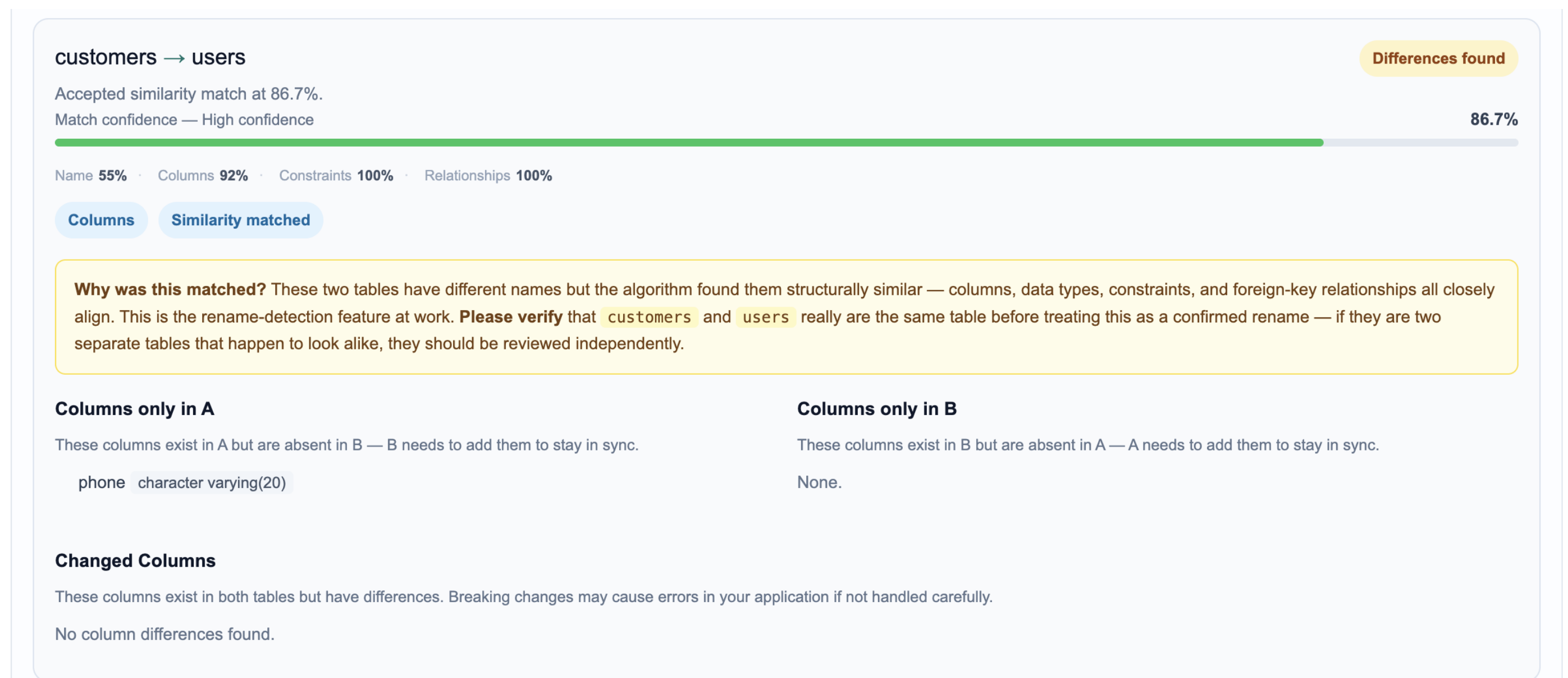
03

Apply Pipeline

- Pull script history from GitHub
- Detect version gaps, queue missing versions in order
- Execute scripts sequentially via pg Node.js driver
- Update in-database version table on success

Key Results

- All Constraint types compared: PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK, EXCLUDE, nullability etc
- Sem-Ver bump levels auto-detected from change severity (Major / Minor / Patch)
- Similarity scoring correctly identifies renamed entities: no false drop-and-create pairs
- Full lifecycle in one interface: generate → review → push → pull → apply



Registry

Stored in PostgreSQL table public.scripts

[Register Script](#)[Refresh](#)

sync_hr_v1_to_hr_v2

1 version

BREAKING

latest v1.0.0

[Hide](#)

v1.0.0

May 7, 2026, 11:59 PM · 47 SQL lines

BREAKING

[Copy SQL](#)[Push](#)

Sync postgres.hr_v1 to match postgres.hr_v2

```
-- Migration: postgres.hr_v2 → postgres.hr_v1
-- Direction: bring B in sync with A
-- Statements: 21 (10 breaking · 2 safe · 9 info)
-- =====

-- [BREAKING] Rename table "leave_requests" → "time_off" (70.6% similarity – verify this is a rename and not
ALTER TABLE "hr_v1"."leave_requests" RENAME TO "time_off";

-- [SAFE] Add column "end_date" (date) to "employee_roles"
ALTER TABLE "hr_v1"."employee_roles" ADD COLUMN "end_date" date;
```

Conclusions

This tool automates the full schema drift lifecycle from detection to deployment, replacing a fragile manual process with a structured, auditable workflow. The similarity algorithm handles renamed entities and constraint drift.

GitHub integration provides a versioned record of all migration scripts. SemVer auto-bumping makes correct versioning the default, not the exception.

Recommendations

Extend engine support to MySQL and SQLite. Integrate with CI/CD pipelines for automated drift detection on every deployment. Add role-based access control and audit trails for team environments. Implement AI-assisted migration review to flag destructive changes before execution, and rollback/dry-run functionality for safer production deployments.